

客服Agent实战系列

构建智能客服系统

AI技术专栏

2026年

目录

1. 客服Agent实战系列 01
2. 客服Agent实战系列 02
3. 客服Agent实战系列 03
4. 客服Agent实战系列 04
5. 客服Agent实战系列 05
6. 客服Agent实战系列 06
7. 客服Agent实战系列 07
8. 客服Agent实战系列 08
9. 客服Agent实战系列 09
10. 客服Agent实战系列 10
11. 客服Agent实战系列 11
12. 客服Agent实战系列 12
13. 客服Agent实战系列 13
14. 客服Agent实战系列 14

1. 客服Agent实战系列 01

客服Agent实战系列 01

客服Agent实战系列 01

项目初始化：把 Agent 开发系列的知识用起来

前面我们用了29篇文章讲

Agent

开发：感知、规划、行动、记忆、工具、执行引擎.....现在，是时候把这些知识

用起来

了。本系列是

Agent开发系列

的实战篇——用代码实现一个真正可运行的智能客服系统。

1

为什么选客服场景？三个理由

2026年，AI 落地最成熟的场景有三个：客服、代码助手、知识问答。为什么选客服？

理由

具体说明

需求最明确

FAQ查询、订单查询、转人工——场景边界清晰，不会"什么都想做"

效果可量化

解决率、响应时间、人工转接率——每个指标都能算ROI

技术最成熟

意图识别、RAG检索、多轮对话——2026年这些已经是"标配"而非"创新"

学完这个系列，你得到的不是一个"玩具项目"，而是一个

可以真正上线运行的客服系统

。

2

与 Agent 开发系列的联动

Agent 开发系列讲了原理，本系列负责落地：

Agent开发系列

客服实战系列

03：最小可行Agent

02：客服Bot定义

05：感知模块

03：意图识别插件

11：RAG检索

04：知识库插件

07：核心循环

05：对话管理

13：Tool接口

08：工单系统

17：执行引擎

09：Web接口

项目结构：

```
customer-service-agent/
├── config/           # 配置文件
|   ├── base.yaml    # 基础配置
|   └── customer_service.yaml # 客服专用
├── data/            # 业务数据
|   ├── faq.json      # FAQ知识库
|   └── intents.json  # 意图定义
├── plugins/         # 客服插件
|   └── intent_recognition/ # 意图识别
```

```
| |—— knowledge_base/    # 知识库检索
| |—— emotion_analysis/  # 情感分析
| |—— ticket_system/     # 工单系统
|—— src/                 # 核心逻辑
|—— web/                 # Web接口
|—— examples/           # 示例代码
```

3

客服Agent架构总览

图1：客服Agent核心架构

4

大模型选择：云端 vs 本地（2026年6月）

2026年的大模型格局已经很清晰了：

云端追求效果，本地追求隐私和成本

。

类型

推荐模型

适用场景

云端-智谱

GLM-5.1-flash

性价比高，中文强，客服首选

云端-阿里

通义千问3-turbo

极速响应，企业级支持

云端-DeepSeek

DeepSeek V3

代码能力强，技术客服

本地-Ollama

Qwen3-8B

中文最佳，无需API Key

本地-vLLM

GLM5-9B

高性能推理，生产环境

base.yaml

llm:

云端：智谱AI GLM-5.1（推荐）

provider: zhipu

model: glm-5.1-flash

api_key: \${ZHIPU_API_KEY}

本地：Ollama（无需API Key）

provider: ollama

model: qwen3:8b

base_url: http://localhost:11434

本地部署三步走：

1.

安装 Ollama：

curl -fsSL https://ollama.com/install.sh | sh

2.

拉取模型：

ollama pull qwen3:8b

3.

启动服务：

ollama serve

本地部署的优势：

数据不出本地、无API费用、可离线运行

。

5

数据准备：三类数据就够了

客服系统不需要复的数据模型，三类数据足矣：

图2：数据流转过程

数据类型

文件

说明

FAQ知识库

faq.json

常见问题及标准答案

意图定义

intents.json

用户意图分类（查询、投诉、转人工等）

实体词典

entities.json

订单号、产品名等关键实体

faq.json 示例：

```
{  
  
  "id": "faq_001",  
  
  "question": "如何修改密码？",  
  
  "answer": "1.登录账户 2.进入账户设置 3.点击修改密码...",  
  
  "keywords": ["密码", "修改密码", "忘记密码"]  
  
}
```

快速验证：5分钟跑起来

图3：快速验证流程

1. 安装依赖

```
pip install -r requirements.txt
```

2. 配置环境变量（云端）

```
export ZHIPU_API_KEY="your-key"
```

或本地部署（无需API Key）

```
ollama pull qwen3:8b && ollama serve
```

3. 验证配置

```
python -c "import yaml; print(yaml.safe_load(open('config/base.yaml')))"
```

4. 测试大模型

```
python -c "
```

```
from zhipuai import ZhipuAI
```

```
client = ZhipuAI()
```

```
print(client.chat.completions.create(
```

```
model='glm-5.1-flash',
```

```
messages=[{'role': 'user', 'content': '你好'}]
```

```
).choices[0].message.content)
```

```
"
```

总结：从原理到实践

本篇要点：

1.

原理已讲过

：Agent开发系列（01-29）覆盖了所有核心概念

2.

现在要落地

：本系列负责把原理变成可运行的代码

3.

场景要聚焦

：客服场景边界清晰，效果可量化

4.

模型要选对

：云端追求效果，本地追求隐私

5.

先跑起来

：5分钟验证，不要陷入配置地狱

下一篇，我们来写

客服Bot的核心：System Prompt设计

——如何让AI真正"懂"客服业务。

客服Agent实战系列

01 · 项目初始化 ← 你在这里

02 · Bot定义：System Prompt设计

03 · 意图识别插件

04 · 知识库插件

... 共14篇

前置系列：

Agent开发系列（01-29）

2. 客服Agent实战系列 02

客服Agent实战系列 02

客服Agent实战系列 02

Bot定义：System Prompt设计

上一篇我们搭好了项目结构，这一篇来写

客服Bot的核心：System Prompt

。很多人觉得 Prompt 就是写几句描述，其实不然——好的 System Prompt 是 Bot 的"灵魂"，决定了它怎么理解任务、怎么回答用户、怎么处理边界情况。

1

System Prompt 是什么？为什么重要？

System Prompt 是给大模型的"角色设定"，告诉它：

设定内容

作用

角色身份

你是谁？（客服助手、技术专家、销售顾问）

能力边界

你能做什么？不能做什么？

行为约束

怎么回答？语气、风格、格式

知识来源

从哪里获取信息？（知识库、工具）

异常处理

遇到不知道的问题怎么办？

图：System Prompt 在对话中的位置

2

客服Bot的System Prompt设计

一个完整的客服 System Prompt 包含五个部分：

markdown

客服Bot System Prompt 模板

1. 角色定义

你是一个专业的客服助手，服务于XX公司的用户。

2. 能力说明

你可以：

3. 行为约束

4. 知识来源

5. 异常处理

3

实战：编写客服Bot的Prompt

我们创建一个完整的客服 Prompt 文件：

config/prompts/system_prompt.md：

markdown

智能客服助手

角色定义

你是"小助手"，XX公司的智能客服。

能力范围

你可以做的：

1. FAQ回答：回答产品使用、账户管理等常见问题
2. 订单查询：查询订单状态、物流信息
3. 问题诊断：帮助用户排查常见问题
4. 转人工：当无法解决时，转接人工客服

你不能做的：

1. 处理退款、取消订单（需要人工审核）
2. 修改用户账户信息
3. 回答与客服无关的问题

回答规范

语气风格：

回答格式：

长度控制：

转人工触发条件

以下情况必须转人工：

1. 用户明确要求转人工
2. 用户表达强烈不满或投诉
3. 问题涉及退款、账户修改等敏感操作
4. 连续3次无法回答用户问题

4

Prompt加载与使用

在代码中加载 System Prompt：

图：Bot初始化流程

src/bot.py：

```
python
```

```
import
```

```
os
```

```
from
```

```
pathlib
```

```
import
```

```
Path
```

```
import
```

```
yaml
```

```
class
```

```
CustomerServiceBot
```

```
:
```

```
def
```

```
__init__
```

```
(
```

```
self
```

```
, config_path: str =
```

```
"config/customer_service.yaml"
```

```
):
```

```
self
```

```
.config =
```

```
self
```

```
._load_config(config_path)
```

```
self
```

```
.system_prompt =  
  
self  
  
._load_prompt()  
  
def  
    _load_config  
  
    (  
  
    self  
  
    , config_path: str) -> dict:  
  
    with  
  
    open(config_path,  
  
        'r'  
  
        , encoding=  
  
        'utf-8'  
  
    )  
  
    as  
  
    f:  
  
    return  
  
    yaml.safe_load(f)  
  
def  
    _load_prompt  
  
    (  
  
    self  
  
    ) -> str:  
  
    prompt_path = Path(  
  
        "config/prompts/system_prompt.md"  
  
    )
```

```
if
prompt_path.exists():
return
prompt_path.read_text(encoding=
"utf-8"
)
return
"你是一个客服助手。"
def
chat
(
self
, user_input: str, history: list =
None
) -> str:
messages = [{
"role"
:
"system"
,
"content"
:
self
.system_prompt}]
if
history:
```

```
messages.extend(history)
```

```
messages.append({
```

```
"role"
```

```
:
```

```
"user"
```

```
,
```

```
"content"
```

```
: user_input})
```

```
return
```

```
self
```

```
._call_llm(messages)
```

```
5
```

测试Prompt效果

```
python
```

```
# 测试客服Bot
```

```
from
```

```
src.bot
```

```
import
```

```
CustomerServiceBot
```

```
bot =
```

```
CustomerServiceBot
```

```
()
```

```
# 测试1：FAQ问题
```

```
print
```

```
(bot.chat(
```

```
"如何修改密码？"
```


))

测试2：情绪激动

print

(bot.chat(

"你们的服务太差了！我要投诉！"

))

测试3：无关问题

print

(bot.chat(

"今天天气怎么样？"

))

6

Prompt优化技巧

三个优化原则：

1.

具体而非抽象

：不要说"礼貌回答"，要说"使用'您'称呼，先表达理解"

2.

给出示例

：写几个好的回答示例，让模型学习风格

3.

迭代测试

：写完Prompt后测试10–20个场景，根据效果调整

常见问题

优化方法

回答太长

在Prompt中限制字数："不超过200字"

语气不统一

给出示例："好的回答示例：您好，这个问题可以这样解决..."

乱回答无关问题

明确边界："只回答产品、订单相关问题"

不转人工

列出触发条件："投诉、退款、连续3次无法回答"

7

总结：Prompt是Bot的灵魂

本篇要点：

1.

System Prompt决定Bot的性格

：角色、能力、约束、知识、异常处理

2.

五个部分不能少

：角色定义、能力说明、行为约束、知识来源、异常处理

3.

具体而非抽象

：给出明确指令和示例

4.

迭代测试

：写完Prompt后测试多场景，持续优化

5.

代码分离

：Prompt存独立文件，方便修改

下一篇，我们来写

意图识别插件

——如何让Bot理解用户想做什么。

客服Agent实战系列

01 · 项目初始化

02 · Bot定义：System Prompt设计 ← 你在这里

03 · 意图识别插件

04 · 知识库插件

... 共14篇

前置系列：

Agent开发系列 05：感知模块

3. 客服Agent实战系列 03

客服Agent实战系列 03

客服Agent实战系列 03

意图识别插件：让Bot理解用户想做什么

上一篇我们设计了System Prompt，告诉Bot"你是谁"。这一篇来解决更关键的问题：

Bot怎么知道用户想做什么？

用户说"我的订单在哪"，Bot要知道这是"订单查询"；用户说"我要投诉"，Bot要知道这是"转人工"。这就是意图识别——客服Bot的"耳朵"。

1

什么是意图识别？为什么重要？

意图识别（Intent Recognition）是Agent感知模块的核心功能。它的作用是：

从用户的自然语言中，提取出用户的意图

。

用户输入

识别出的意图

后续处理

"如何修改密码"

faq_query

从FAQ知识库检索答案

"我的订单在哪"

order_query

调用订单查询接口

"我要投诉"

complaint

立即转人工客服

"你好"

greeting

返回问候语

"今天天气怎么样"

unknown

礼貌拒绝，引导回客服话题

图：意图识别流程示意

意图识别的重要性：

三个关键作用：

1.

决定后续流程

：不同意图走不同的处理路径

2.

提高效率

：不用每次都让大模型"猜"，意图识别可以快速分流

3.

降低成本

：意图识别可以用轻量模型，比每次调用大模型便宜

2

客服场景的意图定义

客服场景的意图相对固定，我们定义了7个核心意图：

json

[

{

"id"

:

"intent_001"

,

"name"

:

"faq_query"

,

"description"

:

"FAQ问题查询"

,

"examples"

:[

"如何修改密码"

,

"怎么申请退款"

,

"配送要多久"

]

},

{

"id"

:

"intent_002"

,

"name"

:

```
"order_query"

,

"description"

:

"订单查询"

,

"examples"

: [

"我的订单在哪里"

,

"查一下订单"

,

"订单状态"

]

},

{

"id"

:

"intent_003"

,

"name"

:

"human_handoff"

,

"description"

:
```

"转人工"

,

"examples"

:[

"转人工"

,

"我要人工客服"

,

"帮我转人工"

]

},

{

"id"

:

"intent_004"

,

"name"

:

"complaint"

,

"description"

:

"投诉"

,

"examples"

:[

"我要投诉"

,

"服务太差了"

,

"投诉你们"

]

},

{

"id"

:

"intent_005"

,

"name"

:

"greeting"

,

"description"

:

"问候"

,

"examples"

:[

"你好"

,

"在吗"

,

"有人吗"

]

},

{

"id"

:

"intent_006"

,

"name"

:

"thanks"

,

"description"

:

"感谢"

,

"examples"

:[

"谢谢"

,

"感谢"

,

"好的谢谢"

]

},

{

```
"id"

:

"intent_007"

,

"name"

:

"unknown"

,

"description"

:

"未识别意图"

,

"examples"

: []

}

]

3
```

意图识别的实现方式

2026年，意图识别有三种主流方式：

方式

优点

缺点

适用场景

关键词匹配

速度快、成本低

准确率低、需要维护关键词库

简单场景、快速分流

小模型分类

准确率高、成本低

需要训练数据、维护模型

意图固定的场景

大模型识别

准确率最高、灵活

成本高、速度慢

复杂场景、意图不固定

图：Bot根据意图分流处理

客服场景我们选择

小模型分类 + 大模型兜底

的混合方案：

混合方案的优势：

1. 常见意图（FAQ、订单查询）用小模型快速识别，成本低

2. 复杂意图（投诉、情绪识别）用大模型，准确率高

3. 未识别意图用大模型兜底，不会漏掉

4

实战：实现意图识别插件

我们创建意图识别插件：

plugins/intent_recognition/plugin.py：

```
python
```

```
import
```

```
json
```

```
from
```

```
pathlib
```

```
import
Path
from
typing
import
Dict, List
class
IntentRecognitionPlugin
:
"""意图识别插件"""
def
__init__
(
self
, intents_file: str =
"data/intents.json"
):
self
.intents =
self
._load_intents(intents_file)
self
.keyword_map =
self
._build_keyword_map()
def
```

```
_load_intents

(

self

, intents_file: str) -> List[Dict]:

"""加载意图定义"""

path = Path(intents_file)

if

path.exists():

with

open(path,

'r'

, encoding=

'utf-8'

)

as

f:

return

json.load(f)

return

[]

def

_build_keyword_map

(

self

) -> Dict[str, str]:

"""构建关键词映射表"""
```

```
keyword_map = {}

for
intent

in

self

.intents:

for

example

in

intent.get(

"examples"

, []):

# 提取关键词（简化版）

keywords = example.split()

for

kw

in

keywords:

if

kw

not

in

keyword_map:

keyword_map[kw] = intent[

"name"

]
```

```
return

keyword_map

def

recognize

(

self

, user_input: str) -> Dict:

"""识别用户意图"""

# 第一步：关键词快速匹配

intent =

self

._keyword_match(user_input)

# 第二步：如果关键词匹配失败，用大模型识别

if

intent ==

"unknown"

:

intent =

self

._llm_recognize(user_input)

return

{

"intent"

: intent,

"confidence"

:


```


0.9

,

"original_input"

: user_input

}

def

_keyword_match

(

self

, user_input: str) -> str:

"""关键词匹配"""

words = user_input.split()

for

word

in

words:

if

word

in

self

.keyword_map:

return

self

.keyword_map[word]

return

"unknown"

```
def
    _llm_recognize
(
    self
, user_input: str) -> str:
    """大模型识别（简化版）"""
    # 实际项目中会调用大模型API
    # 这里简化为返回unknown
    return
    "unknown"
5
```

集成到Bot中

将意图识别插件集成到客服Bot：

src/bot.py（更新版）：

```
python
from
plugins.intent_recognition.plugin
import
IntentRecognitionPlugin
class
    CustomerServiceBot
:
    def
        __init__
(
            self
```

```
, config_path: str =  
"config/customer_service.yaml"  
):  
  
self  
  
.config =  
  
self  
  
._load_config(config_path)  
  
self  
  
.system_prompt =  
  
self  
  
._load_prompt()  
  
self  
  
.intent_plugin = IntentRecognitionPlugin()  
  
def  
  
chat  
  
(  
  
self  
  
, user_input: str, history: list =  
  
None  
  
) -> str:  
  
# 先识别意图  
  
intent_result =  
  
self  
  
.intent_plugin.recognize(user_input)  
  
intent = intent_result[  
  
"intent"
```

```
]
```

```
# 根据意图分流处理
```

```
if
```

```
intent ==
```

```
"faq_query"
```

```
:
```

```
return
```

```
self
```

```
._handle_faq(user_input)
```

```
elif
```

```
intent ==
```

```
"order_query"
```

```
:
```

```
return
```

```
self
```

```
._handle_order_query(user_input)
```

```
elif
```

```
intent ==
```

```
"complaint"
```

```
:
```

```
return
```

```
self
```

```
._handle_complaint(user_input)
```

```
elif
```

```
intent ==
```

```
"human_handoff"
```

```
:  
  
return  
  
self  
  
._handle_handoff(user_input)  
  
else  
  
:  
  
# 其他意图走大模型  
  
return  
  
self  
  
._call_llm_with_context(user_input, history)  
  
def  
  
_handle_faq  
  
(  
  
self  
  
, user_input: str) -> str:  
  
# 下一篇会实现知识库插件  
  
return  
  
"正在从FAQ知识库检索答案..."  
  
def  
  
_handle_order_query  
  
(  
  
self  
  
, user_input: str) -> str:  
  
# 后续会实现订单查询插件  
  
return  
  
"正在查询您的订单..."
```

```
def
_handle_complaint
(
self
, user_input: str) -> str:
return
"非常抱歉给您带来不便。我立即为您转接人工客服处理。"
```

```
def
_handle_handoff
(
self
, user_input: str) -> str:
return
"好的，正在为您转接人工客服..."
```

6

测试意图识别效果

python

测试意图识别

from

plugins.intent_recognition.plugin

import

IntentRecognitionPlugin

plugin =

IntentRecognitionPlugin

()

测试各种意图

```
print

(plugin.recognize(

"如何修改密码"

))

# 输出 : {'intent': 'faq_query', 'confidence': 0.9}

print

(plugin.recognize(

"我的订单在哪"

))

# 输出 : {'intent': 'order_query', 'confidence': 0.9}

print

(plugin.recognize(

"我要投诉"

))

# 输出 : {'intent': 'complaint', 'confidence': 0.9}

print

(plugin.recognize(

"你好"

))

# 输出 : {'intent': 'greeting', 'confidence': 0.9}

print

(plugin.recognize(

"今天天气怎么样"

))

# 输出 : {'intent': 'unknown', 'confidence': 0.9}
```

意图识别的优化方向

三个优化方向：

1.

增加训练数据

：收集更多用户对话，训练小模型分类器

2.

上下文理解

：结合历史对话，识别多轮意图

3.

置信度阈值

：低置信度的意图用大模型二次确认

优化方法

效果

成本

增加关键词库

提高关键词匹配准确率

低（人工维护）

训练小模型

提高整体准确率到95%+

中（需要数据）

上下文理解

识别多轮意图

高（需要记忆模块）

8

总结：意图识别是Bot的"耳朵"

本篇要点：

1.

意图识别决定分流

：不同意图走不同处理路径

2.

混合方案最优

：关键词快速匹配 + 大模型兜底

3.

意图定义要清晰

：客服场景7个核心意图足够

4.

插件化设计

：意图识别作为独立插件，方便扩展

5.

持续优化

：收集数据，训练小模型，提高准确率

下一篇，我们来写

知识库插件

——如何让Bot从FAQ中找到答案。

客服Agent实战系列

01 · 项目初始化

02 · Bot定义：System Prompt设计

03 · 意图识别插件 ← 你在这里

04 · 知识库插件

... 共14篇

前置系列：

Agent开发系列 05：感知模块

4. 客服Agent实战系列 04

客服Agent实战系列 04

客服Agent实战系列 04

知识库插件：让Bot从FAQ中找到答案

上一篇我们实现了意图识别，Bot知道用户想做什么了。这一篇来解决更核心的问题：

用户问"如何修改密码"，Bot怎么从FAQ知识库中找到答案？

这就是RAG（检索增强生成）——客服Bot的"大脑"。

1

什么是RAG？为什么重要？

RAG（Retrieval-Augmented

Generation，检索增强生成）是2026年最主流的AI知识问答方案。它的核心思想：

先检索，再生成

。

传统方案

RAG方案

优势

让大模型"记住"所有知识

从知识库检索相关内容，再让大模型生成答案

知识可更新、成本低、准确率高

图：RAG检索流程

RAG的工作流程：

三个步骤：

1.

检索

：从知识库中找到与用户问题相关的文档

2.

增强

：将检索到的文档作为上下文，提供给大模型

3.

生成

：大模型基于上下文生成答案

2

FAQ知识库的结构

客服场景的FAQ知识库相对简单，我们用JSON格式存储：

```
json
[
{
  "id"
:
  "faq_001"
,
  "question"
:
  "如何修改密码？"
,
  "answer"
:
  "您可以通过以下步骤修改密码：\n1.      登录账户\n2.      进入「账户设置」\n3.      点击「修改密码」\n4.      输入原密码和新密码\n5. 点击确认保存"
,
  "category"
:
}
```

"账户相关"

,

"keywords"

:[

"密码"

,

"修改密码"

,

"改密码"

,

"忘记密码"

]

},

{

"id"

:

"faq_002"

,

"question"

:

"如何申请退款？"

,

"answer"

:

"退款申请流程：\n1. 进入「我的订单」\n2. 找到需要退款的订单\n3. 点击「申请退款」\n4.

填写退款原因\n5. 等待审核（1-3个工作日）"

,

"category"

:

"订单相关"

,

"keywords"

:[

"退款"

,

"退货"

,

"申请退款"

]

}

]

FAQ知识库的关键字段：

字段

作用

question

FAQ的标准问题

answer

FAQ的标准答案

keywords

关键词，用于快速检索

category

分类，用于组织管理

3

知识库插件的实现方式

2026年，知识库检索有两种主流方式：

方式

优点

缺点

适用场景

关键词检索

速度快、实现简单

准确率低、需要维护关键词库

FAQ数量少 (<100)

向量检索

准确率高、语义理解

需要向量数据库、成本稍高

FAQ数量多 (>100)

图：知识库检索流程

客服场景我们选择

关键词检索 + 向量检索

的混合方案：

混合方案的优势：

1. 常见问题（关键词匹配）用关键词检索，速度快
2. 复问题（语义理解）用向量检索，准确率高
3. 检索结果融合，取最佳匹配

4

实战：实现知识库插件

我们创建知识库插件：

plugins/knowledge_base/plugin.py：

```
python
import
json
from
pathlib
import
Path
from
typing
import
List, Dict
class
KnowledgeBasePlugin
:
    """知识库插件"""
    def
        __init__(
            self
            , faq_file: str =
                "data/faq.json"
        ):
            self
                .faqs =
                    self
                        ._load_faqs(faq_file)
```

```
self

.keyword_index =

self

._build_keyword_index()

def

_load_faqs

(

self

, faq_file: str) -> List[Dict]:

"""加载FAQ知识库"""

path = Path(faq_file)

if

path.exists():

with

open(path,

'r'

, encoding=

'utf-8'

)

as

f:

return

json.load(f)

return

[]

def
```



```
_build_keyword_index

(

self

) -> Dict[str, List[str]]:

    """构建关键词索引"""

    index = {}

    for

    faq

    in

    self

    .faqs:

    for

    keyword

    in

    faq.get(

    "keywords"

    , []):

    if

    keyword

    not

    in

    index:

    index[keyword] = []

    index[keyword].append(faq[

    "id"

    ])
```

```
return

index

def

search

(

self

, query: str, top_k: int =

3

) -> List[Dict]:

"""检索FAQ"""

# 第一步：关键词检索

keyword_results =

self

._keyword_search(query)

# 第二步：向量检索（如果有向量数据库）

vector_results =

self

._vector_search(query)

# 第三步：融合结果

all_results = keyword_results + vector_results

# 去重并返回top_k

unique_results =

self

._deduplicate(all_results)

return

unique_results[:top_k]
```

```
def
_keyword_search
(
self
, query: str) -> List[Dict]:
"""关键词检索"""
results = []
query_words = query.split()
for
word
in
query_words:
if
word
in
self
.keyword_index:
faq_ids =
self
.keyword_index[word]
for
faq_id
in
faq_ids:
faq =
self
```

```
._get_faq_by_id(faq_id)

if

faq:

results.append({

    "faq"

    : faq,

    "score"

    :

    1.0

    ,

    "source"

    :

    "keyword"

})

return

results

def

_vector_search

(

    self

    , query: str) -> List[Dict]:

    """向量检索（简化版）"""

    # 实际项目中会调用向量数据库API

    # 这里简化为返回空列表

    return

    []
```

```
def
_get_faq_by_id
(
self
, faq_id: str) -> Dict:
    """根据ID获取FAQ"""
    for
    faq
    in
    self
    .faqs:
    if
    faq[
    "id"
    ] == faq_id:
    return
    faq
    return
    None
def
_deduplicate
(
self
, results: List[Dict]) -> List[Dict]:
    """去重"""
    seen = set()
```

```
unique = []

for
result

in

results:

faq_id = result[

"faq"

][

"id"

]

if

faq_id

not

in

seen:

seen.add(faq_id)

unique.append(result)

return

unique

5
```

集成到Bot中

将知识库插件集成到客服Bot：

src/bot.py（更新版）：

```
python
```

```
from
```

```
plugins.knowledge_base.plugin
```

```
import

KnowledgeBasePlugin

class

CustomerServiceBot

:

def

__init__

(

self

, config_path: str =

"config/customer_service.yaml"

):

self

.config =

self

._load_config(config_path)

self

.system_prompt =

self

._load_prompt()

self

.intent_plugin = IntentRecognitionPlugin()

self

.kb_plugin = KnowledgeBasePlugin()

def

_handle_faq
```

```
(
self
, user_input: str) -> str:
# 从知识库检索FAQ
results =
self
.kb_plugin.search(user_input, top_k=
1
)
if
results:
faq = results[
0
][
"faq"
]
# 直接返回FAQ答案
return
faq[
"answer"
]
# 如果没找到，走大模型
return
"抱歉，我暂时无法回答这个问题。建议您转人工客服获取帮助。"
```

6

测试知识库效果


```
python

# 测试知识库检索

from

plugins.knowledge_base.plugin

import

KnowledgeBasePlugin

kb =

KnowledgeBasePlugin

()

# 测试关键词检索

print

(kb.search(

"如何修改密码"

))

# 输出：找到faq_001，返回修改密码的答案

print

(kb.search(

"申请退款"

))

# 输出：找到faq_002，返回退款流程

print

(kb.search(

"配送多久"

))

# 输出：找到faq_003，返回配送时间说明

# 测试完整Bot
```

```
from
src.bot

import

CustomerServiceBot

bot =

CustomerServiceBot

()

print

(bot.chat(

"如何修改密码"

))

# 输出：您可以通过以下步骤修改密码...

7
```

RAG的优化方向

三个优化方向：

1.

向量检索

：使用向量数据库（如Pinecone、Weaviate）进行语义检索

2.

重排序

：使用CrossEncoder对检索结果进行重排序，提高准确率

3.

答案生成

：不只是返回FAQ原文，让大模型基于FAQ生成更自然的答案

优化方法

效果

成本

增加关键词库

提高关键词检索覆盖率

低（人工维护）

向量检索

提高语义理解能力

中（需要向量数据库）

答案生成

答案更自然、更个性化

中（调用大模型）

8

总结：知识库是Bot的"大脑"

本篇要点：

1.

RAG是主流方案

：先检索，再生成，知识可更新

2.

FAQ知识库结构简单

：question、answer、keywords、category

3.

混合检索最优

：关键词快速检索 + 向量语义检索

4.

插件化设计

：知识库作为独立插件，方便扩展

5.

持续优化

：增加FAQ数量、引入向量检索、优化答案生成

下一篇，我们来写

对话管理

——如何让Bot记住对话历史，处理多轮对话。

客服Agent实战系列

01 · 项目初始化

02 · Bot定义：System Prompt设计

03 · 意图识别插件

04 · 知识库插件 ← 你在这里

05 · 对话管理

... 共14篇

前置系列：

Agent开发系列 11：RAG检索

5. 客服Agent实战系列 05

客服Agent实战系列 05

客服Agent实战系列 05

对话管理：让Bot记住对话历史

上一篇我们实现了知识库检索，Bot能从FAQ中找到答案了。这一篇来解决多轮对话的核心问题：

用户说"我的订单在哪"，Bot回答后，用户又说"那配送要多久"，Bot怎么知道"那"指的是什么？

这就是对话管理——客服Bot的"记忆"。

1

什么是对话管理？为什么重要？

对话管理（Dialog Management）是Agent记忆模块的核心功能。它的作用是：

记住对话历史，理解上下文，处理多轮对话

。

单轮对话

多轮对话

区别

"如何修改密码" → 返回答案

"我的订单在哪" → 返回订单信息

"那配送要多久" → 返回配送时间

第二句依赖第一句的上下文

图：多轮对话流程

对话管理的重要性：

三个关键作用：

1.

上下文理解

：理解"那"、"这个"等指代词的含义

2.

意图延续

：记住上一轮的意图，处理后续相关问题

3.

状态管理

：记录对话状态（如等待用户输入订单号）

2

对话历史的结构

对话历史用消息列表（Messages）存储，每条消息包含角色和内容：

```
json
[
  {
    "role"
    :
    "system"
    ,
    "content"
    :
    "你是XX公司的智能客服..."
  },
  {
    "role"
    :
    "user"
    ,
    "content"
```

:

"我的订单在哪"

},

{

"role"

:

"assistant"

,

"content"

:

"您的订单12345正在配送中..."

},

{

"role"

:

"user"

,

"content"

:

"那配送要多久"

}

]

消息角色的含义：

角色

作用

system

System Prompt，设定Bot的角色和行为

user

用户的输入

assistant

Bot的回复

3

对话管理的实现方式

2026年，对话管理有三种主流方式：

方式

优点

缺点

适用场景

全量记忆

简单、完整

占用内存、成本高

对话轮数少（<10轮）

滑动窗口

节省内存、成本低

可能丢失重要上下文

对话轮数多（>10轮）

摘要记忆

保留关键信息、高效

需要额外处理、可能遗漏细节

长对话（>20轮）

图：对话管理流程

客服场景我们选择

滑动窗口 + 关键信息提取

的混合方案：

混合方案的优势：

1. 保留最近5轮对话，确保上下文连贯
2. 提取关键信息（订单号、用户ID），避免重复询问
3. 超过5轮的对话，用摘要压缩

4

实战：实现对话管理

我们创建对话管理模块：

src/dialog_manager.py：

```
python
```

```
from
```

```
typing
```

```
import
```

```
List, Dict
```

```
class
```

```
DialogManager
```

```
:
```

```
"""对话管理器"""
```

```
def
```

```
__init__
```

```
(
```

```
self
```

```
, max_history: int =
```

```
10
```

```
):
```

```
self

.history: List[Dict] = []

self

.max_history = max_history

self

.key_info: Dict = {}

def

add_message

(

self

, role: str, content: str):

"""添加消息到历史"""

self

.history.append({

"role"

: role,

"content"

: content

})

# 提取关键信息

self

._extract_key_info(content)

# 滑动窗口：保留最近max_history条消息

if

len(

self
```

```
.history) >

self

.max_history:

self

.history =

self

.history[-

self

.max_history:]

def

get_history

(

self

) -> List[Dict]:

"""获取对话历史"""

return

self

.history

def

get_key_info

(

self

) -> Dict:

"""获取关键信息"""

return

self
```

```
.key_info

def
_extract_key_info

(

self

, content: str):

"""提取关键信息（简化版）"""

# 实际项目中会用正则或大模型提取

# 这里简化为提取订单号

import

re

order_pattern = r

'订单[号]?[:\s]*(\d+)'

match = re.search(order_pattern, content)

if

match:

self

.key_info[

"order_id"

] = match.group(

1

)

def

clear

(

self
```

):

"""清空对话历史"""

self

.history = []

self

.key_info = {}

5

集成到Bot中

将对话管理集成到客服Bot：

src/bot.py（更新版）：

python

from

src.dialog_manager

import

DialogManager

class

CustomerServiceBot

:

def

__init__

(

self

, config_path: str =

"config/customer_service.yaml"

):

self

```
.config =

self

._load_config(config_path)

self

.system_prompt =

self

._load_prompt()

self

.intent_plugin = IntentRecognitionPlugin()

self

.kb_plugin = KnowledgeBasePlugin()

self

.dialog_manager = DialogManager(max_history=

10

)

def

chat

(

self

, user_input: str) -> str:

# 识别意图

intent_result =

self

.intent_plugin.recognize(user_input)

intent = intent_result[

"intent"
```

```
]
```

```
# 添加用户消息到历史
```

```
self
```

```
.dialog_manager.add_message(
```

```
"user"
```

```
, user_input)
```

```
# 根据意图处理
```

```
if
```

```
intent ==
```

```
"faq_query"
```

```
:
```

```
response =
```

```
self
```

```
._handle_faq(user_input)
```

```
elif
```

```
intent ==
```

```
"order_query"
```

```
:
```

```
response =
```

```
self
```

```
._handle_order_query(user_input)
```

```
else
```

```
:
```

```
response =
```

```
self
```

```
._call_llm_with_context(user_input)
```

```
# 添加Bot回复到历史
```

```
self
```

```
.dialog_manager.add_message(
```

```
"assistant"
```

```
, response)
```

```
return
```

```
response
```

```
def
```

```
_call_llm_with_context
```

```
(
```

```
self
```

```
, user_input: str) -> str:
```

```
# 构建消息列表
```

```
messages = [{
```

```
"role"
```

```
:
```

```
"system"
```

```
,
```

```
"content"
```

```
:
```

```
self
```

```
.system_prompt}]
```

```
# 添加对话历史
```

```
history =
```

```
self
```

```
.dialog_manager.get_history()
```



```
messages.extend(history)

# 添加关键信息提示

key_info =

self

.dialog_manager.get_key_info()

if

key_info:

context_msg = f

"已知信息：{key_info}"

messages.append({

"role"

:

"system"

,

"content"

: context_msg})

# 调用大模型

return

self

._call_llm(messages)

6

测试对话管理效果

python

# 测试多轮对话

from

src.bot
```

```
import
CustomerServiceBot

bot =
CustomerServiceBot

()

# 第一轮

print

(bot.chat(

"我的订单12345在哪"

))

# 输出：您的订单12345正在配送中...

# 第二轮（依赖第一轮上下文）

print

(bot.chat(

"那配送要多久"

))

# 输出：您的订单12345预计2-3天送达...

# Bot记住了订单号12345

# 第三轮

print

(bot.chat(

"可以修改地址吗"

))

# 输出：订单12345配送中，无法修改地址...

# Bot仍然记得订单号
```

对话管理的优化方向

三个优化方向：

1.

智能摘要

：超过10轮对话，用大模型生成摘要，保留关键信息

2.

意图追踪

：记录每轮意图，识别意图转换（如从查询转为投诉）

3.

状态机

：用状态机管理对话流程（如等待订单号输入状态）

优化方法

效果

成本

智能摘要

节省内存，保留关键信息

中（调用大模型）

意图追踪

识别意图转换，提高准确率

低（记录意图）

状态机

规范对话流程，提高效率

中（设计状态）

8

总结：对话管理是Bot的"记忆"

本篇要点：

1.

对话管理记住历史

：理解上下文，处理多轮对话

2.

消息列表是核心

：system、user、assistant三种角色

3.

滑动窗口最优

：保留最近10轮，提取关键信息

4.

关键信息提取

：订单号、用户ID等，避免重复询问

5.

持续优化

：智能摘要、意图追踪、状态机

下一篇，我们来写

情感分析插件

——如何让Bot识别用户情绪，及时转人工。

客服Agent实战系列

01 · 项目初始化

02 · Bot定义：System Prompt设计

03 · 意图识别插件

04 · 知识库插件

05 · 对话管理 ← 你在这里

06 · 情感分析插件

... 共14篇

前置系列：

Agent开发系列 06：记忆模块

6. 客服Agent实战系列 06

客服Agent实战系列 06

客服Agent实战系列 06

情感分析插件：让Bot识别用户情绪

上一篇我们实现了对话管理，Bot能记住对话历史了。这一篇来解决客服场景的关键问题：

用户说"你们的服务太差了！我要投诉！"，Bot怎么识别这是愤怒情绪，及时转人工？

这就是情感分析——客服Bot的"情商"。

1

什么是情感分析？为什么重要？

情感分析（Sentiment Analysis）是Agent感知模块的高级功能。它的作用是：

识别用户情绪，及时响应负面情绪

。

用户输入

识别的情感

Bot的处理

"你好"

中性

正常回复

"谢谢"

正面

礼貌感谢

"你们的服务太差了"

负面（愤怒）

安抚+转人工

"我等了很久了"

负面（不耐烦）

道歉+加快处理

图：情感分析流程

情感分析的重要性：

三个关键作用：

1.

及时转人工

：识别愤怒情绪，立即转人工客服

2.

调整语气

：识别负面情绪，用更温和的语气回复

3.

预警机制

：识别情绪变化趋势，提前干预

2

情感等级的定义

客服场景的情感分为5个等级：

```
json
[
{
"level"
:
1
,
"name"
:

```

"正面"

,

"keywords"

:[

"谢谢"

,

"感谢"

,

"好的"

,

"满意"

],

"response"

:

"礼貌感谢"

},

{

"level"

:

2

,

"name"

:

"中性"

,

"keywords"


```
: [  
  "你好"  
  ,  
  "在吗"  
  ,  
  "请问"  
],  
"response"  
  
:  
  "正常回复"  
},  
{  
  "level"  
:  
  3  
  ,  
  "name"  
:  
  "轻微负面"  
  ,  
  "keywords"  
:  
  "有点慢"  
  ,  
  "不太方便"  
},
```

"response"

:

"道歉+改进建议"

},

{

"level"

:

4

,

"name"

:

"负面"

,

"keywords"

:[

"太慢了"

,

"等了很久"

,

"不满意"

],

"response"

:

"道歉+加快处理"

},

{

"level"

:

5

,

"name"

:

"愤怒"

,

"keywords"

:[

"投诉"

,

"太差了"

,

"垃圾"

,

"骗子"

],

"response"

:

"立即转人工"

}

]

情感等级的处理策略：

等级

情感

处理策略

Level 1

正面

礼貌感谢："感谢您的支持！"

Level 2

中性

正常回复，按意图处理

Level 3

轻微负面

道歉："抱歉给您带来不便..."

Level 4

负面

道歉+加快："非常抱歉，我立即为您处理..."

Level 5

愤怒

立即转人工："非常抱歉，正在为您转接人工客服..."

3

情感分析的实现方式

2026年，情感分析有三种主流方式：

方式

优点

缺点

适用场景

关键词匹配

速度快、成本低

准确率低、需要维护关键词库

简单场景、快速分流

小模型分类

准确率高、成本低

需要训练数据、维护模型

情感固定的场景

大模型识别

准确率最高、灵活

成本高、速度慢

复场景、情感多变

图：情感等级处理流程

客服场景我们选择

关键词匹配 + 大模型识别

的混合方案：

混合方案的优势：

1. 常见情绪词（"投诉"、"太差了"）用关键词匹配，速度快

2. 复情绪表达用大模型识别，准确率高

3. 情感等级 ≥ 4 ，立即转人工

4

实战：实现情感分析插件

我们创建情感分析插件：

plugins/sentiment_analysis/plugin.py：

```
python
```

```
import
```

```
json
```

```
from
```

```
pathlib
```

```
import
Path
from
typing
import
Dict
class
SentimentAnalysisPlugin
:
    """情感分析插件"""
    def
    __init__
    (
    self
    , sentiment_file: str =
    "data/sentiments.json"
    ):
    self
    .sentiments =
    self
    ._load_sentiments(sentiment_file)
    self
    .keyword_map =
    self
    ._build_keyword_map()
    def
```

```
_load_sentiments

(

self

, sentiment_file: str) -> list:

"""加载情感定义"""

path = Path(sentiment_file)

if

path.exists():

with

open(path,

'r'

, encoding=

'utf-8'

)

as

f:

return

json.load(f)

return

[]

def

_build_keyword_map

(

self

) -> Dict[str, int]:

"""构建关键词映射表"""
```

```
keyword_map = {}

for
sentiment

in

self

.sentiments:

for

keyword

in

sentiment.get(

"keywords"

, []):

keyword_map[keyword] = sentiment[

"level"

]

return

keyword_map

def

analyze

(

self

, user_input: str) -> Dict:

"""分析用户情感"""

# 第一步：关键词匹配

sentiment_level =

self
```



```
._keyword_match(user_input)

# 第二步：如果关键词匹配失败，用大模型识别

if

sentiment_level ==

0

:

sentiment_level =

self

._llm_analyze(user_input)

# 获取情感信息

sentiment =

self

._get_sentiment_by_level(sentiment_level)

return

{

"level"

: sentiment_level,

"name"

: sentiment.get(

"name"

,

"未知"

),

"response_strategy"

: sentiment.get(

"response"
```

```
,  
"正常回复"  
,  
"need_handoff"  
: sentiment_level >=  
4  
}  
  
def  
_keyword_match  
(  
self  
, user_input: str) -> int:  
    """关键词匹配"""  
    max_level =  
    0  
    words = user_input.split()  
    for  
    word  
    in  
    words:  
        if  
        word  
        in  
        self  
        .keyword_map:  
            level =
```

```
self

.keyword_map[word]

max_level = max(max_level, level)

return

max_level

def

_llm_analyze

(

self

, user_input: str) -> int:

"""大模型分析（简化版）"""

# 实际项目中会调用大模型API

# 这里简化为返回中性

return

2

def

_get_sentiment_by_level

(

self

, level: int) -> Dict:

"""根据等级获取情感信息"""

for

sentiment

in

self

.sentiments:
```

```
if
sentiment[
"level"
] == level:
return
sentiment
return
{
"level"
:
2
,
"name"
:
"中性"
,
"response"
:
"正常回复"
}
5
```

集成到Bot中

将情感分析插件集成到客服Bot：

src/bot.py（更新版）：

```
python
```

```
from
```

```
plugins.sentiment_analysis.plugin

import

SentimentAnalysisPlugin

class

CustomerServiceBot

:

def

__init__

(

self

, config_path: str =

"config/customer_service.yaml"

):

self

.config =

self

._load_config(config_path)

self

.system_prompt =

self

._load_prompt()

self

.intent_plugin = IntentRecognitionPlugin()

self

.kb_plugin = KnowledgeBasePlugin()

self
```

```
.dialog_manager = DialogManager(max_history=
10
)
self
.sentiment_plugin = SentimentAnalysisPlugin()
def
chat
(
self
, user_input: str) -> str:
# 先分析情感
sentiment_result =
self
.sentiment_plugin.analyze(user_input)
# 如果情感等级≥4，立即转人工
if
sentiment_result[
"need_handoff"
]:
return
"非常抱歉给您带来不便。我立即为您转接人工客服处理。"
# 正常处理意图
intent_result =
self
.intent_plugin.recognize(user_input)
# 根据情感调整回复语气
```

```
response =  
  
self  
  
._process_intent(intent_result, user_input)
```

```
# 根据情感等级调整语气
```

```
if
```

```
sentiment_result[
```

```
"level"
```

```
] ==
```

```
3
```

```
:
```

```
response =
```

```
"抱歉给您带来不便。"
```

```
+ response
```

```
return
```

```
response
```

```
6
```

```
测试情感分析效果
```

```
python
```

```
# 测试情感分析
```

```
from
```

```
plugins.sentiment_analysis.plugin
```

```
import
```

```
SentimentAnalysisPlugin
```

```
plugin =
```

```
SentimentAnalysisPlugin
```

```
()
```

测试正面情绪

print

(plugin.analyze(

"谢谢"

))

输出：{'level': 1, 'name': '正面', 'need_handoff': False}

测试中性情绪

print

(plugin.analyze(

"你好"

))

输出：{'level': 2, 'name': '中性', 'need_handoff': False}

测试负面情绪

print

(plugin.analyze(

"太慢了"

))

输出：{'level': 4, 'name': '负面', 'need_handoff': True}

测试愤怒情绪

print

(plugin.analyze(

"我要投诉"

))

输出：{'level': 5, 'name': '愤怒', 'need_handoff': True}

测试完整Bot

from


```
src.bot
```

```
import
```

```
CustomerServiceBot
```

```
bot =
```

```
CustomerServiceBot
```

```
()
```

```
print
```

```
(bot.chat(
```

```
"你们的服务太差了！"
```

```
))
```

```
# 输出：非常抱歉给您带来不便。我立即为您转接人工客服处理。
```

```
7
```

情感分析的优化方向

三个优化方向：

1.

情绪追踪

：记录情绪变化趋势，识别情绪升级（如从"有点慢"到"太慢了"）

2.

个性化回复

：根据用户历史情绪，调整回复策略

3.

预警机制

：情绪等级 ≥ 3 时，提前预警，准备转人工

优化方法

效果

成本

情绪追踪

识别情绪升级，提前干预

低（记录情绪）

个性化回复

提高用户满意度

中（分析历史）

预警机制

减少投诉率

低（设置阈值）

8

总结：情感分析是Bot的"情商"

本篇要点：

1.

情感分析识别情绪

：正面、中性、轻微负面、负面、愤怒5个等级

2.

关键词匹配最快

：常见情绪词快速识别

3.

大模型识别最准

：复情绪表达准确识别

4.

等级≥4转人工

：负面和愤怒情绪立即转人工

5.

调整回复语气

：根据情感等级调整语气

下一篇，我们来写

订单查询插件

——如何让Bot查询订单状态。

客服Agent实战系列

01 · 项目初始化

02 · Bot定义：System Prompt设计

03 · 意图识别插件

04 · 知识库插件

05 · 对话管理

06 · 情感分析插件 ← 你在这里

07 · 订单查询插件

... 共14篇

前置系列：

Agent开发系列 05：感知模块

7. 客服Agent实战系列 07

客服Agent实战系列 07

客服Agent实战系列 07

订单查询插件：让Bot真正"办事"

前面我们做了意图识别、知识库、情感分析。现在来做一个真正能"办事"的插件——订单查询。用户问"我的订单到哪了"，Bot得能查数据库、返回真实状态。

订单查询插件的核心价值：

连接业务系统

，把大模型的"语言理解"转化为"业务操作"。

典型客服场景中的订单查询需求：

查询订单状态（待付款/已发货/配送中/已送达）

查询物流信息（快递单号、预计送达时间）

查询订单详情（商品、金额、收货地址）

查询历史订单

图：订单查询流程

1

订单查询插件的设计

我们将订单查询插件设计为

独立的插件模块

，与Bot解耦，便于维护和扩展。

插件的核心功能：

功能

说明

订单查询

根据订单号/手机号查询订单

物流追踪

查询物流状态和预计送达时间

订单列表

查询用户的历史订单

结果格式化

将数据转化为自然语言

图：订单查询插件架构

2

数据源选择：直连数据库 vs 调用API

订单数据通常存在业务系统中，有两种常见接入方式：

方式

优点

缺点

适用场景

直连数据库

速度快、数据实时

权限管理复杂、耦合高

内部系统、只读查询

调用订单API

解耦、安全、易维护

需要API支持、有网络开销

对外服务、跨系统

实战建议：

优先使用API方式

，更安全、更易维护。如果没有API，短期可直连只读数据库，但要做好权限控制。

3

实现订单查询插件

新建插件：plugins/order_query/plugin.py

```
python
```

```
import
```

```
re
```

```
from
```

```
typing
```

```
import
```

```
Dict,
```

```
Optional
```

```
class
```

```
OrderQueryPlugin
```

```
:
```

```
def
```

```
__init__
```

```
(self,
```

```
db_connection
```

```
None
```

```
):
```

```
self
```

```
.
```

```
db
```

```
db_connection
```

```
def
```

```
query
```

```
(self,  
  
user_input:  
  
str)
```

```
Dict:  
  
order_id
```

```
self  
  
.  
  
_extract_order_id(user_input)  
  
phone
```

```
self  
  
.  
  
_extract_phone(user_input)
```

```
if  
  
not  
  
order_id
```

```
and  
  
not
```

```
phone:  
  
return
```

```
{  
  
"success"  
  
:
```

```
False
```

```
,  
  
"need_more_info"
```

:

True

,

"message"

:

"请提供订单号或手机号，我帮您查询。"

}

orders

self

.

_search_orders(order_id,

phone)

if

not

orders:

return

{

"success"

:

False

,

"message"

:

"未找到相关订单，请确认信息是否正确。"

}

return


```
{  
  
    "success"  
  
    :  
  
    True  
  
    ,  
  
    "orders"  
  
    :  
  
    orders,  
  
    "formatted"  
  
    :  
  
    self  
  
    .  
  
    _format_orders(orders)  
  
}  
  
def  
  
    _extract_order_id  
  
(self,  
  
text:  
  
str)  
  
Optional[str]:  
  
    match  
  
  
    re  
  
    .  
  
    search(  
  
r'订单[号]?[:\s]*(\w{6,})'
```

```
,  
text)  
  
return  
  
match  
  
.  
  
group(  
1  
)  
  
if  
  
match  
  
else  
  
None  
  
def  
  
_extract_phone  
  
(self,  
  
text:  
  
str)  
  
Optional[str]:  
  
match  
  
  
re  
  
.  
  
search(  
  
r'1[3-9]\d{9}'  
  
,  
  
text)
```

return

match

.

group(

0

)

if

match

else

None

def

_search_orders

(self,

order_id,

phone):

实际项目中，这里会调用数据库或API

简化示例

return

[

{

"order_id"

:

"20240601001"

,

"status"

:

"配送中"

,

"items"

:

[

"玫瑰花束"

],

"tracking"

:

"SF123456"

}

]

def

_format_orders

(self,

orders):

parts

[]

for

o

in

orders:

parts

.

append(

f"订单 {

```
o[
'order_id'
]
}: {
o[
'status'
]
}"
)
return
"
\n
"
.
join(parts)
4
```

集成到Bot中

将订单查询插件集成到Bot的chat流程中：

```
python
def
chat
(self,
user_input:
str)

str:

# 情感分析
```

sentiment

self

.

sentiment_plugin

.

analyze(user_input)

if

sentiment[

"need_handoff"

]:

return

"非常抱歉，我立即为您转人工客服。"

意图识别

intent

self

.

intent_plugin

.

recognize(user_input)

订单查询意图

if

intent[

"intent"

]

"order_query"

```
:  
  
result  
  
self  
  
.  
  
order_plugin  
  
.  
  
query(user_input)  
  
if  
  
result[  
  
"need_more_info"  
  
]:  
  
return  
  
result[  
  
"message"  
  
]  
  
if  
  
result[  
  
"success"  
  
]:  
  
return  
  
result[  
  
"formatted"  
  
]  
  
return  
  
result[  
  
"message"
```

```
]
```

```
# 其他意图处理...
```

```
response
```

```
self
```

```
.
```

```
_process_intent(intent,
```

```
user_input)
```

```
return
```

```
response
```

```
5
```

```
测试订单查询
```

```
python
```

```
# 测试订单查询
```

```
from
```

```
plugins.order_query.plugin
```

```
import
```

```
OrderQueryPlugin
```

```
plugin
```

```
OrderQueryPlugin()
```

```
# 有订单号
```

```
result
```

```
plugin
```

```
.
```

```
query(
```

```
"我的订单号20240601001到哪了"
```


)

print(result[

"formatted"

])

输出：订单 20240601001：配送中

无订单号

result

plugin

.

query(

"我的订单到哪了"

)

print(result[

"message"

])

输出：请提供订单号或手机号，我帮您查询。

手机号查询

result

plugin

.

query(

"用手机号13800138000查订单"

)

6

订单查询的实战要点

信息提取

：用正则提取订单号、手机号，支持多种格式

多轮补全

：信息不足时主动询问，而不是返回失败

结果格式化

：返回的是数据，展示给用户的是自然语言

异常处理

：数据库异常时给用户友好提示，不暴露技术细节

安全控制

：只能查询当前用户的订单，不能越权

核心心法：

插件是Bot的"手和脚"

，让Bot不仅能聊天，还能真正"办事"。

7

总结与下一篇

本篇要点：

订单查询插件连接业务系统

优先使用API方式接入

信息不足时多轮补全

结果要格式化为自然语言

下一篇，我们进入

多轮对话进阶

——处理槽位填充、上下文指代、任务型对话等复杂场景。

客服Agent实战系列

01 · 项目初始化

02 · Bot定义：System Prompt设计

- 03 · 意图识别插件
- 04 · 知识库插件
- 05 · 对话管理
- 06 · 情感分析插件
- 07 · 订单查询插件 ← 你在这里
- 08 · 多轮对话进阶
- 09 · 知识库增强与向量检索
- 10 · Function Calling
- 11 · 多Agent协作
- 12 · 用户记忆与个性化
- 13 · 效果评估与持续优化
- 14 · 生产级架构总结

8. 客服Agent实战系列 08

客服Agent实战系列 08

客服Agent实战系列 08

多轮对话进阶：槽位填充与指代消解

前面我们做了基础的对话管理（滑动窗口、历史保留）。但实际客服场景比"记住上一句"复：

"那个订单"指代什么？"刚才说的那个"是什么意思？

本篇来解决多轮对话的核心难题。

多轮对话进阶要解决三大核心问题：

指代消解

："它"、"那个"、"刚才"指的是什么？

槽位填充

：逐步收集任务所需的信息（如订票：时间/地点/人数）

任务推进

：根据当前状态决定下一步该做什么

图：槽位填充示例：订花场景

1

槽位填充：任务型对话的核心

槽位填充（Slot Filling）是任务型对话的核心思想：

把一个任务拆解为若干必要信息（槽位），逐步收集。

槽位填充的基本流程：

定义任务所需的槽位（如订票：时间、地点、人数）

检查当前已填充的槽位

如果槽位缺失，主动向用户询问

所有槽位填满后，执行任务

示例：订花场景的槽位：花型、数量、配送地址、配送时间。用户的每一轮对话都可能填充一个或多个槽位。

2

指代消解：让Bot理解"它"和"那个"

指代消解是多轮对话的难点。用户常使用代词指代之前提到的事物。

指代消解的常用方法：

方法

说明

示例

最近提及优先

指代最近提到的实体

"它怎么样？" → 最近的订单

语义匹配

根据上下文语义推断

"那个可以退吗？" → 商品

关键词回指

用关键词回指历史信息

"刚才说的那个订单" → 之前的订单

图：上下文窗口：滑动窗口 + 关键信息

3

任务状态机：让对话有明确方向

任务型对话通常遵循一定的流程，用

状态机

来描述最清晰。

示例：订票任务的状态机

状态

含义

触发条件

INIT

初始状态

识别为订票意图

ASK_TIME

询问时间

缺少时间信息

ASK_PLACE

询问地点

缺少地点信息

CONFIRM

确认信息

所有槽位已填充

DONE

任务完成

用户确认

4

实现：扩展DialogManager

扩展 src/dialog_manager.py，增加槽位和状态追踪：

python

class

DialogManager

:

def

__init__

(self,

max_history

10

):

self

.

history

[]

self

.

max_history

max_history

self

.

slots

{}

当前任务的槽位

self

.

state

"INIT"

当前状态

self

.

entities

```
[]
```

```
# 提到的实体列表
```

```
def
```

```
add_message
```

```
(self,
```

```
role,
```

```
content):
```

```
self
```

```
.
```

```
history
```

```
.
```

```
append({
```

```
"role"
```

```
:
```

```
role,
```

```
"content"
```

```
:
```

```
content})
```

```
self
```

```
.
```

```
_extract_entities(content)
```

```
if
```

```
len(self
```

```
.
```

```
history)
```

```
>
```



```
self
.
max_history:

self
.
history

self
.
history[

self
.
max_history:]

def
_extract_entities

(self,
text):

# 提取订单号、日期、地点等实体

pass

def
resolve_reference

(self,
pronoun):

# 指代消解：返回代词指向的实体

for
entity
```

```
in
reversed(self
.
entities):
if
entity[
"text"
]

pronoun:
return
entity
return
self
.
entities[

1
]
if
self
.
entities
else
None
def
fill_slot
```

```
(self,  
  
slot_name,  
  
value):  
  
self  
  
.   
  
slots[slot_name]
```

value

def

missing_slots

(self,

required_slots):

return

[s

for

s

in

required_slots

if

s

not

in

self

.

slots]

5

多轮对话的测试示例

```
python
```

```
# 多轮对话示例
```

```
bot
```

```
CustomerServiceBot()
```

```
# 用户发起任务
```

```
print(bot
```

```
.
```

```
chat(
```

```
"我想订一束花"
```

```
))
```

```
# 输出：好的！请告诉我您要什么花？
```

```
# 填充槽位
```

```
print(bot
```

```
.
```

```
chat(
```

```
"玫瑰花"
```

```
))
```

```
# 输出：好的，玫瑰花。要多少朵？
```

```
print(bot
```

```
.
```

```
chat(
```

```
"99朵"
```

```
))
```

```
# 输出：好的，99朵玫瑰花。送到哪里？
```

```
# 指代消解
```

```
print(bot
```

.

chat(

"它什么时候能到？"

))

Bot根据上下文，理解"它"指玫瑰花订单

输出：预计明天下午送达。

6

实战经验与注意事项

保持对话流自然

：不要机械地一个问题接一个问题

允许多槽位一次填充

："我要99朵玫瑰明天送到国贸"一次填多个槽

支持用户修改

："刚才说错了，要康乃馨"能回退修改

超时保护

：长时间无对话，槽位清空或保存

打断处理

：对话中突然问其他问题，支持跳转到其他任务

核心心法：

好的多轮对话像真人客服——有记忆、有耐心、能理解指代

。

7

总结与下一篇

本篇要点：

槽位填充：任务型对话的核心

指代消解：让Bot理解代词

状态机：让对话有明确方向

扩展DialogManager支持多轮

下一篇，我们进入

知识库增强

——用Embedding和向量检索让Bot真正"理解"文档，而不是只做关键词匹配。

客服Agent实战系列

01 · 项目初始化

02 · Bot定义：System Prompt设计

03 · 意图识别插件

04 · 知识库插件

05 · 对话管理

06 · 情感分析插件

07 · 订单查询插件

08 · 多轮对话进阶 ← 你在这里

09 · 知识库增强与向量检索

10 · Function Calling

11 · 多Agent协作

12 · 用户记忆与个性化

13 · 效果评估与持续优化

14 · 生产级架构总结

9. 客服Agent实战系列 09

客服Agent实战系列 09

客服Agent实战系列 09

知识库增强：Embedding与向量检索

前面我们做了基础的知识库（关键词匹配FAQ）。但用户问"我的快递到哪了"，即使FAQ里有"物流查询"的条目，关键词匹配也可能找不到。

因为关键词匹配不理解"快递"就是"物流"。

本篇用Embedding + 向量检索让知识库"理解"语义。

关键词检索 vs 向量检索的核心区别：

对比维度

关键词检索

向量检索

匹配方式

字面匹配

语义匹配

同义词

不支持

支持

理解深度

浅

深

计算开销

低

高

适用场景

精确查找

模糊语义查询

图：关键词检索 vs 向量检索

1

Embedding是什么？

Embedding是将文本（词、句子、段落）转化为

高维向量

的过程。转化后的向量具有"语义相似则距离近"的特性。

Embedding的工作流程：

文本 → Tokenizer 切分为Token

Token → Embedding模型 → 向量

向量间可用余弦相似度衡量语义距离

示例："我的快递到了吗" 与 "物流状态查询" 的Embedding向量会非常接近，即使字面完全不同。

2

向量检索RAG的整体流程

RAG (Retrieval-Augmented Generation) 是

检索增强生成

的缩写，是当前大模型应用的核心架构。

图：RAG检索增强生成流程

RAG的四个步骤：

索引阶段：将所有文档Embedding后存入向量数据库

检索阶段：用户问题Embedding后查询向量数据库

生成阶段：将检索到的相关文档与问题一起喂给大模型

回答阶段：大模型基于提供的上下文生成答案

3

向量数据库选型

常用的向量数据库对比：

数据库

特点

适用场景

Chroma

轻量级、Python原生

原型开发、小中型项目

Milvus

高性能、分布式

大规模生产环境

FAISS

Facebook开源、高效

研究、单机场景

Pinecone

全托管云服务

不想运维、快速上线

pgvector

PostgreSQL扩展

已有PG的项目

入门推荐：

Chroma

，Python原生、接口简单，几行代码就能跑通。生产环境推荐Milvus或Pinecone。

4

实现：增强知识库插件

先安装依赖：

bash

pip

install

chromadb

sentence-transformers

实现 plugins/knowledge_base/vector_plugin.py :

python

import

chromadb

from

sentence_transformers

import

SentenceTransformer

class

VectorKnowledgeBasePlugin

:

def

__init__

(self,

persist_dir

"data/vector_db"

):

self

.

model

SentenceTransformer(

"shibing624/text2vec"

)

self

.

client

chromadb

.

PersistentClient(path

persist_dir)

self

.

collection

self

.

client

.

get_or_create_collection(

name

"faq"

,

metadata

{

"hnsw:space"

:

```
"cosine"
```

```
}
```

```
)
```

```
def
```

```
add_documents
```

```
(self,
```

```
documents):
```

```
ids
```

```
[
```

```
    f"doc_{
```

```
        i
```

```
    }"
```

```
for
```

```
    i
```

```
in
```

```
range(len(documents))]
```

```
embeddings
```

```
[self
```

```
    .
```

```
model
```

```
    .
```

```
encode(d[
```

```
    "content"
```

```
]])
```

```
    .
```

tolist()

for

d

in

documents]

self

.

collection

.

add(

ids

ids,

embeddings

embeddings,

documents

[d[

"content"

]

for

d

in

documents],

metadatas

[d

```
.  
  
get(  
  
"meta"  
  
,  
  
{})  
  
for  
  
d  
  
in  
  
documents]  
  
)  
  
def  
  
search  
  
(self,  
  
query,  
  
top_k  
  
  
3  
  
):  
  
query_embedding  
  
  
self  
  
.  
  
model  
  
.  
  
encode(query)  
  
.  
  
tolist()
```

```
results

self

.

collection

.

query(

query_embeddings

[query_embedding],

n_results

top_k

)

return

results[

"documents"

][

0

]

if

results[

"documents"

]

else

[]

5

构建索引：入库FAQ数据
```

```
python

import

json

from

plugins.knowledge_base.vector_plugin

import

VectorKnowledgeBasePlugin

# 加载FAQ数据

with

open(

"data/faq.json"

,

"r"

,

encoding

"utf-8"

)

as

f:

    faqs

    json

    .

    load(f)

# 初始化插件并入库

kb
```


VectorKnowledgeBasePlugin()

documents

```
[
{
  "content"
:
faq[
  "question"
]
+
" "
+
faq[
  "answer"
],
"meta"
:
{
  "category"
:
faq
.
get(
  "category"
,
  ""
```

```
    }}  
  
    for  
  
    faq  
  
    in  
  
    faqs  
  
    ]  
  
    kb  
  
    .  
  
    add_documents(documents)  
  
    # 测试检索  
  
    results  
  
    kb  
  
    .  
  
    search(  
  
        "我的快递到了吗"  
  
    )  
  
    for  
  
    r  
  
    in  
  
    results:  
  
        print(r)  
  
    # 输出最相关的FAQ：物流查询、快递状态等
```

6

混合方案：关键词 + 向量检索

关键词检索和向量检索各有所长，

实际项目中通常用混合方案

:

关键词精确匹配优先（如订单号、产品型号）

关键词未命中时，降级到向量检索

向量检索后，用关键词做精确过滤

python

def

search

(self,

query,

top_k

3

):

第一步：关键词精确匹配

kw_result

self

.

kb_plugin

.

search_by_keyword(query)

if

kw_result:

return

kw_result

第二步：向量语义检索

vec_results

```
self
.
vector_kb
.
search(query,
top_k)

# 第三步：关键词过滤

filtered

[r
for
r
in
vec_results
if
self
.
_keyword_filter(query,
r)]

return
filtered
or
vec_results

7
```

总结与下一篇

本篇要点：

Embedding将文本转为向量

RAG检索增强生成是核心架构

向量数据库选型：Chroma/Milvus/FAISS

混合方案兼顾精确与语义

下一篇，我们进入

Function Calling

——让大模型自己决定调用哪个工具，而不是我们手动写 if-else。

客服Agent实战系列

01 · 项目初始化

02 · Bot定义：System Prompt设计

03 · 意图识别插件

04 · 知识库插件

05 · 对话管理

06 · 情感分析插件

07 · 订单查询插件

08 · 多轮对话进阶

09 · 知识库增强与向量检索 ← 你在这里

10 · Function Calling

11 · 多Agent协作

12 · 用户记忆与个性化

13 · 效果评估与持续优化

14 · 生产级架构总结

10. 客服Agent实战系列 10

客服Agent实战系列 10

客服Agent实战系列 10

Function Calling：让大模型自己决定用什么工具

前面我们手动写了 `if intent == "order_query"` 来路由。但客服场景越来越复，意图越来越多，if-else 会失控。

Function Calling 让大模型自己决定"调用哪个工具"。

这是Agent的核心能力之一。

Function Calling（函数调用）的核心思想：

给大模型一组"工具"（函数）的描述，让它根据用户输入决定调用哪个工具。

图：Function Calling 工作流程

1

为什么需要Function Calling

手动路由的问题：

意图数量多了以后，if-else 代码难以维护

边界情况处理困难（一句话可能对应多个意图）

新增工具需要改代码、重新部署

Function Calling的优势：

大模型自主决策，减少硬编码

自然扩展工具，只需添加描述

支持多工具组合（一个请求调用多个工具）

图：可用工具集合

2

工具定义：以JSON Schema的形式

大模型需要知道每个工具的

名称、参数、描述

。我们用JSON格式定义：

```
json
[
{
"type"
:
"function"
,
"function"
: {
"name"
:
"get_order_status"
,
"description"
:
"查询订单状态，输入订单号，返回订单当前状态"
,
"parameters"
: {
"type"
:
"object"
,

```

"properties"

: {

"order_id"

: {

"type"

:

"string"

,

"description"

:

"订单号，如20240601001"

}

},

"required"

: [

"order_id"

]

}

}

},

{

"type"

:

"function"

,

"function"


```
: {  
  
  "name"  
  
  :  
  
  "get_weather"  
  
  ,  
  
  "description"  
  
  :  
  
  "查询指定城市的当前天气"  
  
  ,  
  
  "parameters"  
  
  : {  
  
    "type"  
  
    :  
  
    "object"  
  
    ,  
  
    "properties"  
  
    : {  
  
      "city"  
  
      : {  
  
        "type"  
  
        :  
  
        "string"  
  
        ,  
  
        "description"  
  
        :  
  
        "城市名称"
```

```
}  
  
,  
  
"required"  
  
: [  
  
"city"  
  
]  
  
}  
  
}  
  
}  
  
]  
  
3
```

实现Function Calling流程

步骤1：发送请求时带上tools参数

```
python  
  
import  
  
requests  
  
def  
  
call_llm_with_tools  
  
(user_input,  
  
tools):  
  
response  
  
requests  
  
.  
  
post(  
  
"https://api.example.com/v1/chat/completions"  
  
,
```

json

{

"model"

:

"gpt-4o"

,

"messages"

:

[{

"role"

:

"user"

,

"content"

:

user_input}],

"tools"

:

tools

}

)

return

response

.

json([

"choices"

```
]]
```

```
0
```

```
]]
```

```
"message"
```

```
]
```

步骤2：判断是否需要调用工具

```
python
```

```
message
```

```
call_llm_with_tools(
```

```
"我的订单20240601001到哪了"
```

```
,
```

```
tools)
```

```
if
```

```
message
```

```
.
```

```
get(
```

```
"tool_calls"
```

```
):
```

```
# 大模型决定调用工具
```

```
for
```

```
tool_call
```

```
in
```

```
message[
```

```
"tool_calls"
```

```
]:
```

```
tool_name
```

```
tool_call[
    "function"
]
"name"
]

tool_args

json

.

loads(tool_call[
    "function"
]
    "arguments"
])
```

执行工具

result

execute_tool(tool_name,

tool_args)

else

:

大模型直接回复

print(message[

"content"

])

步骤3：执行工具并返回结果

python

```
def
execute_tool

(name,

args):

if

name

"get_order_status"

:

return

order_plugin

.

query(args[

"order_id"

])

elif

name

"get_weather"

:

return

weather_api

.

get(args[

"city"

])

else
```

```
:  
  
return  
  
{  
  
"error"  
  
:  
  
"unknown tool"  
  
}
```

步骤4：将工具结果返回给大模型，生成自然语言回复

```
python  
  
def  
  
chat_with_tools  
  
(user_input,  
  
tools):  
  
# 第一轮：大模型决定是否调用工具  
  
message  
  
  
  
call_llm_with_tools(user_input,  
  
tools)  
  
if  
  
not  
  
message  
  
.  
  
get(  
  
"tool_calls"  
  
):  
  
return  
  
message[
```

"content"

]

执行所有工具调用

messages

[{

"role"

:

"user"

,

"content"

:

user_input}]

messages

.

append(message)

for

tool_call

in

message[

"tool_calls"

]:

result

execute_tool(

tool_call[

"function"


```
][  
  "name"  
],  
json  
.  
loads(tool_call[  
  "function"  
  ][  
    "arguments"  
  ])  
)  
messages  
.  
append({  
  "role"  
  :  
  "tool"  
  ,  
  "tool_call_id"  
  :  
  tool_call[  
    "id"  
  ],  
  "content"  
  :  
  json
```

.

dumps(result)

})

第二轮：带工具结果再次请求

final_response

requests

.

post(

"https://api.example.com/v1/chat/completions"

,

json

{

"model"

:

"gpt-4o"

,

"messages"

:

messages}

)

return

final_response

.

json()[

"choices"

```
]]
```

```
0
```

```
]]
```

```
"message"
```

```
]]
```

```
"content"
```

```
]
```

```
4
```

测试Function Calling

```
python
```

```
# 测试
```

```
tools
```

```
[
```

```
...
```

```
]
```

```
# 定义的工具列表
```

```
# 场景1：订单查询
```

```
print(chat_with_tools(
```

```
"我的订单20240601001到哪了"
```

```
,
```

```
tools))
```

```
# 输出：您的订单20240601001目前正在配送中，预计明天送达。
```

```
# 场景2：天气查询
```

```
print(chat_with_tools(
```

```
"北京今天天气怎么样"
```

```
,
```

```
tools))
```

```
# 输出：北京今天晴，温度25℃，适合出行。
```

```
# 场景3：闲聊（不调用工具）
```

```
print(chat_with_tools(
```

```
"你好，你是谁"
```

```
,
```

```
tools))
```

```
# 输出：我是客服Agent，很高兴为您服务！
```

5

工具设计的最佳实践

描述清晰

：工具描述要让大模型明白"何时用、怎么用"

粒度适中

：工具不要太粗或太细（"查询订单" vs "查询订单的状态/物流/详情"）

参数明确

：每个参数说明清楚类型、含义、示例

错误处理

：工具执行失败时返回友好的错误信息

幂等设计

：同一个工具调用多次结果应一致

关键心法：

把工具当作"大模型的外部能力"

，描述越清晰，大模型用得越准。

6

对比：手动路由 vs Function Calling

维度

手动路由

Function Calling

灵活性

固定流程

动态决策

可扩展性

加意图需改代码

加工具即可

维护成本

高

低

准确性

依赖意图识别

依赖LLM理解

复杂度

低

需处理多轮对话

7

总结与下一篇

本篇要点：

Function Calling让大模型自主选择工具

工具以JSON Schema描述

多轮对话：LLM决策→执行→返回→再生成

工具描述越清晰，调用越准确

下一篇，我们进入

多Agent协作

——当一个Agent处理不过来时，如何让多个Agent分工合作？

客服Agent实战系列

01 · 项目初始化

02 · Bot定义：System Prompt设计

03 · 意图识别插件

04 · 知识库插件

05 · 对话管理

06 · 情感分析插件

07 · 订单查询插件

08 · 多轮对话进阶

09 · 知识库增强与向量检索

10 · Function Calling ← 你在这里

11 · 多Agent协作

12 · 用户记忆与个性化

13 · 效果评估与持续优化

14 · 生产级架构总结

11. 客服Agent实战系列 11

客服Agent实战系列 11

客服Agent实战系列 11

多Agent协作：让多个Agent分工合作

前面我们做了单个客服Agent。但实际业务复：订单、投诉、咨询、售后、物流.....每个领域都有专业知识。

一个Agent包打天下，不如多个专业Agent分工合作。

多Agent协作的核心思想：

让每个Agent专注一件事，由一个"调度者"统一协调

。

图：多Agent路由调度

1

为什么需要多Agent

单Agent的问题：

System Prompt过长：所有场景的规则都塞一个Prompt

知识混：订单知识和售后知识混在一起，容易混淆

难以维护：改一个业务可能影响其他业务

扩展性差：加新业务要改整个Agent

多Agent的优势：

每个Agent职责单一、知识聚焦

独立部署、独立迭代

易于扩展：加业务只需加Agent

组合灵活：不同场景用不同的Agent组合

图：前台-后台Agent协作

2

多Agent的三种协作模式

模式

说明

适用场景

路由模式

一个调度Agent决定把请求分发给哪个Agent

不同业务领域（订单/投诉/咨询）

串行模式

多个Agent按顺序处理，上一个的输出作为下一个的输入

复流程（接待→诊断→处理→回访）

并行模式

多个Agent同时处理，结果合并

多信息查询（同时查订单+物流）

3

实现：路由Agent

调度Agent的核心职责：

理解用户意图

选择最合适的子Agent

将用户请求转发给子Agent

将子Agent的返回结果统一格式化

python

class

RouterAgent

:

def

__init__


```
(self):  
  
    self  
  
    .  
  
    agents  
  
    {  
  
        "chat"  
  
        :  
  
        ChatAgent(),  
  
        "order"  
  
        :  
  
        OrderAgent(),  
  
        "complaint"  
  
        :  
  
        ComplaintAgent(),  
  
        "consult"  
  
        :  
  
        ConsultAgent(),  
  
    }  
  
    def  
  
    route  
  
    (self,  
  
    user_input:  
  
    str)  
  
    str:  
  
    # 1. 用LLM判断应该交给哪个Agent
```

```
target
```

```
self
```

```
.
```

```
_decide_target(user_input)
```

```
# 2. 转发给目标Agent
```

```
agent
```

```
self
```

```
.
```

```
agents[target]
```

```
response
```

```
agent
```

```
.
```

```
handle(user_input)
```

```
return
```

```
response
```

```
def
```

```
_decide_target
```

```
(self,
```

```
user_input):
```

```
prompt
```

```
f"""
```

```
根据用户输入，选择最合适的Agent：
```

只返回Agent名称，不要其他内容。

用户输入：{

user_input

}

"""

调用LLM决策

return

self

.

_call_llm(prompt)

.

strip()

4

子Agent的实现

每个子Agent专注自己的领域：

python

class

OrderAgent

:

def

__init__

(self):

self

.

plugin

OrderQueryPlugin()

self

.

system_prompt

"你是订单查询专家..."

def

handle

(self,

user_input:

str)

str:

result

self

.

plugin

.

query(user_input)

if

result[

"success"

]:

return

result[

"formatted"

]

return

```
result[
```

```
"message"
```

```
]
```

```
class
```

```
ComplaintAgent
```

```
:
```

```
def
```

```
__init__
```

```
(self):
```

```
self
```

```
.
```

```
sentiment
```

```
SentimentAnalysisPlugin()
```

```
def
```

```
handle
```

```
(self,
```

```
user_input:
```

```
str)
```

```
str:
```

```
sentiment
```

```
self
```

```
.
```

```
sentiment
```

```
.
```

```
analyze(user_input)
```

```
if
sentiment[
"need_handoff"
]:
return
"非常抱歉，我立即为您转人工客服处理。"
return
"已记录您的反馈，我们会尽快处理。"
```

5

上下文共享与状态管理

多Agent协作时，

如何让不同Agent共享上下文

？

方案：

集中式上下文：所有Agent共享一个Context对象，由Router管理

传递式上下文：Router将当前对话历史传给子Agent

持久化上下文：会话信息存Redis/数据库，Agent按需读取

python

class

SharedContext

:

def

__init__

(self,

session_id):

self

.

session_id

session_id

self

.

history

[]

self

.

slots

{}

self

.

current_agent

None

def

add

(self,

role,

content):

self

.

history

.

```
append({  
  
    "role"  
  
    :  
  
    role,  
  
    "content"  
  
    :  
  
    content})  
  
def  
  
get_history  
  
(self):  
  
    return  
  
    self  
  
    .  
  
    history[  
  
  
  
    10  
  
    :]  
  
    # 最近10条  
  
def  
  
set_slot  
  
(self,  
  
    key,  
  
    value):  
  
    self  
  
    .  
  
    slots[key]
```


value

6

实战注意事项

职责清晰

：每个Agent的边界要明确，避免重叠

统一语言

：不同Agent的回复风格要一致

平滑切换

：Agent切换时给用户过渡提示

兜底策略

：无法路由时，有兜底Agent处理

监控日志

：记录每个Agent的调用情况，便于排查

关键心法：

多Agent不是银弹，适合复杂业务场景。简单场景单Agent即可

。

7

总结与下一篇

本篇要点：

多Agent三种协作模式：路由/串行/并行

路由Agent负责分发和协调

子Agent职责单一、知识聚焦

共享上下文支持跨Agent状态

下一篇，我们进入

用户记忆与个性化

——让Bot记住每个用户的偏好，提供贴心的个性化服务。

客服Agent实战系列

01 · 项目初始化

02 · Bot定义：System Prompt设计

03 · 意图识别插件

04 · 知识库插件

05 · 对话管理

06 · 情感分析插件

07 · 订单查询插件

08 · 多轮对话进阶

09 · 知识库增强与向量检索

10 · Function Calling

11 · 多Agent协作 ← 你在这里

12 · 用户记忆与个性化

13 · 效果评估与持续优化

14 · 生产级架构总结

12. 客服Agent实战系列 12

客服Agent实战系列 12

客服Agent实战系列 12

用户记忆与个性化：让Bot认识每个用户

前面的Bot都是"失忆"的——换个会话就不认识用户了。但真正的客服应该记住：

张阿姨喜欢玫瑰花，李先生上次投诉过物流，王总是VIP客户

。本篇让Bot拥有"记忆"。

记忆系统的两个层次：

短期记忆

：当前会话的上下文（已在对话管理中实现）

长期记忆

：跨会话的用户画像、历史偏好

图：记忆系统架构

1

用户画像：Bot对用户的"了解"

用户画像（User Profile）是长期记忆的核心，

结构化地记录用户的关键信息

。

客服场景的用户画像包含：

类别

内容

示例

基本信息

昵称、性别、地区

张先生、北京

兴趣偏好

喜欢的产品、品类

喜欢玫瑰花

历史行为

历史订单、投诉记录

上次投诉物流

情感特征

常见情绪、沟通风格

耐心、理性

等级标签

VIP、新客户、流失风险

VIP客户

图：用户画像结构

2

记忆的提取：从对话中自动学习

用户画像不是手工录入的，而是

从对话中自动提取和更新

的。

记忆提取的触发时机：

每次对话结束后

用户主动提供信息时

订单、投诉等关键事件发生时

python

class

MemoryManager

:

```
def
__init__
(self,
db):
self
.
db

db

def
extract_and_update
(self,
user_id,
dialogue):
# 1. 提取实体

entities

self
.
_extract_entities(dialogue)

# 2. 更新用户画像

profile

self
.
get_profile(user_id)

for
entity
```

```
in
entities:
if
entity[
"type"
]

"preference"
:
profile
.
setdefault(
"preferences"
,
[])
if
entity[
"value"
]
not
in
profile[
"preferences"
]:
profile[
"preferences"
]
```

```
.  
  
append(entity[  
  
"value"  
  
])
```

3. 保存更新

```
self  
  
.  
  
save_profile(user_id,  
  
profile)  
  
def  
  
_extract_entities  
  
(self,  
  
dialogue):  
  
# 用LLM分析对话，提取关键实体  
  
# 返回：[{"type": "preference", "value": "玫瑰花"}, ...]  
  
pass  
  
3
```

记忆的应用：让服务更贴心

场景1：个性化问候

```
python  
  
def  
  
greet_user  
  
(profile):  
  
if  
  
profile  
  
.  
  

```

```
get(

"level"

)

"VIP"

:

return

"王总您好，好久不见！有什么可以帮您？"

if

profile

.

get(

"nickname"

):

return

f"{

profile[

'nickname'

]

}您好，很高兴再次见到您！"

return

"您好，很高兴为您服务！"
```

场景2：智能推荐

```
python

def

recommend

(profile):
```



```
if
    "玫瑰花"

in
    profile

    .

    get(
        "preferences"
    ,
    []):

    return
        "根据您的喜好，为您推荐新款玫瑰花束"

if
    profile

    .

    get(
        "recent_order"
    ):

    return
        "您上次购买的商品现在有优惠活动"

    return
        "为您推荐本周热销产品"
```

场景3：语气和服务等级调整

```
python

def
    adjust_tone

(profile):
```

if

profile

.

get(

"level"

)

"VIP"

:

return

"尊敬的VIP客户"

if

profile

.

get(

"complaint_history"

):

return

"非常抱歉之前的不便，我们会更加用心"

return

"您好"

4

记忆的存储方案

方案

优点

缺点

适用场景

JSON文件

简单

不适合生产

原型验证

Redis

快、支持过期

需运维

缓存、短期记忆

MySQL

持久化、可靠

查询慢

长期画像

MongoDB

灵活、JSON友好

需运维

复画像

实战建议：

Redis存短期会话，MySQL/MongoDB存长期画像

，冷热分离。

5

隐私与安全

涉及用户数据，必须考隐私和安全：

用户授权

：明确告知用户哪些信息会被记录

数据脱敏

：存储时对敏感字段加密

访问控制

：用户只能查看自己的画像

删除机制

：支持用户一键清除记忆

数据最小化

：只收集必要的信息

6

个性化的边界

不是所有场景都适合个性化。过度个性化可能让用户感到不适：

首次对话不要暴露"我知道你"的感觉

敏感问题（投诉、退款）不要用个性化语气

公共场景下不要提及用户的私人信息

让用户感觉到"贴心"而不是"被监视"

7

总结与下一篇

本篇要点：

用户画像：Bot对用户的"了解"

自动提取和更新记忆

三大应用场景：问候、推荐、语气

隐私安全是重中之重

下一篇，我们进入

效果评估与持续优化

——如何衡量一个Bot的好坏？如何持续改进？

客服Agent实战系列

01 · 项目初始化

02 · Bot定义：System Prompt设计

- 03 · 意图识别插件
- 04 · 知识库插件
- 05 · 对话管理
- 06 · 情感分析插件
- 07 · 订单查询插件
- 08 · 多轮对话进阶
- 09 · 知识库增强与向量检索
- 10 · Function Calling
- 11 · 多Agent协作
- 12 · 用户记忆与个性化 ← 你在这里
- 13 · 效果评估与持续优化
- 14 · 生产级架构总结

13. 客服Agent实战系列 13

客服Agent实战系列 13

客服Agent实战系列 13

效果评估与持续优化

前面我们实现了各种能力。但问题来了：

这个Bot好用吗？用户满意吗？问题解决了吗？

本篇来解决"怎么衡量一个Bot的好坏"。没有度量，就没有改进。

客服Agent的效果评估可以从

五大维度

进行：

图：客服Agent核心指标体系

1

核心指标详解

指标

计算方式

目标值

意图识别准确率

正确识别的轮数/总轮数

≥90%

问题解决率

一次对话解决的比例

≥85%

转人工率

转人工次数/总对话数

≤10%

平均响应时长

从用户输入到Bot回复的时间

$\leq 2s$

用户满意度

用户反馈的满意度评分

$\geq 4.5/5$

指标之间的关系：

意图识别准确率是基础，直接影响问题解决率

问题解决率是核心业务指标

转人工率是成本指标，越低越好

响应时长影响用户体验

用户满意度是最终评价

2

数据采集：日志是金矿

效果评估的基础是

数据

。每次对话都要记录详细的日志：

python

import

logging

import

json

logger

logging

.

getLogger(

```
"agent"

)

def

log_dialogue

(session_id,

user_input,

bot_response,

metadata):

log_entry

{

"session_id"

:

session_id,

"user_input"

:

user_input,

"bot_response"

:

bot_response,

"timestamp"

:

datetime

.

now()

.

isoformat(),
```



```
"metadata"
```

```
:
```

```
metadata
```

```
# 意图、情感、耗时等
```

```
}
```

```
logger
```

```
.
```

```
info(json
```

```
.
```

```
dumps(log_entry,
```

```
ensure_ascii
```

```
False
```

```
))
```

日志应包含：

用户输入和Bot回复

识别的意图和情感

使用的插件和耗时

是否转人工、是否解决问题

用户反馈（如果有）

3

离线评估：用标注数据跑分

离线评估用

标注好的测试集

跑分，衡量各项指标。

```
python
```

```
def
```

```
evaluate_intent_accuracy
```

```
(test_set,
```

```
bot):
```

```
correct
```

```
0
```

```
total
```

```
0
```

```
for
```

```
item
```

```
in
```

```
test_set:
```

```
input_text
```

```
item[
```

```
"input"
```

```
]
```

```
expected_intent
```

```
item[
```

```
"intent"
```

```
]
```

```
result
```

```
bot
```

```
.
```

```
recognize_intent(input_text)
```

```
if
```

```
result[
    "intent"
]

expected_intent:

correct

+=

1

total

+=

1

accuracy

correct

/

total

print(

f"意图识别准确率：{

accuracy

:.2%}"

)

return

accuracy
```

测试集的构建：

覆盖所有意图类型

每个意图至少50条样本

包含典型和边界场景

包含多种表达方式和口语化表达

4

在线评估：真实用户的反馈

离线评估再准，也比不过真实用户的反馈。

上线后要持续收集用户反馈

。

在线评估的方法：

点赞/点踩按钮：最简单的满意度收集

多轮反馈：解决了吗？满意吗？

对话后问卷调查

工单关联：对话是否最终产生了工单

python

def

collect_feedback

(session_id,

is_satisfied):

feedback

{

"session_id"

:

session_id,

"is_satisfied"

:

is_satisfied,

True/False

"timestamp"

:

datetime

.

now()

.

isoformat()

}

save_to_db(feedback)

5

持续优化：数据驱动的迭代

优化是一个持续的循环：

图：数据驱动的优化循环

优化的典型路径：

分析日志，找出转人工的高频场景

分析点踩的对话，定位问题

补充知识库或优化插件

AB测试对比优化效果

全量上线并持续监控

核心心法：

数据→分析→优化→验证

，形成闭环。

6

AB测试：科学对比优化效果

优化不能靠感觉，要用

AB测试

科学对比。

将用户随机分成A、B两组

A组使用旧版本，B组使用新版本

运行一段时间后比较核心指标

如果B组指标更好，则全量上线

AB测试的注意事项：

样本量要足够大（至少几百用户）

测试时间要足够长（至少一周）

两组用户的分布要一致

关注核心指标，不要被次要指标干扰

7

总结与下一篇

本篇要点：

五大核心指标：准确率/解决率/转人工率/响应时长/满意度

数据采集是评估的基础

离线+在线结合的评估体系

数据驱动的持续优化循环

下一篇，我们来做一个

生产级客服Agent架构的总结

，把前面的所有能力整合起来，看看一个真正能用的客服Agent长什么样。

客服Agent实战系列

01 · 项目初始化

02 · Bot定义：System Prompt设计

03 · 意图识别插件

04 · 知识库插件

05 · 对话管理

06 · 情感分析插件

07 · 订单查询插件

08 · 多轮对话进阶

09 · 知识库增强与向量检索

10 · Function Calling

11 · 多Agent协作

12 · 用户记忆与个性化

13 · 效果评估与持续优化 ← 你在这里

14 · 生产级架构总结

14. 客服Agent实战系列 14

客服Agent实战系列 14

客服Agent实战系列 14

生产级客服Agent架构总结

历时14篇，我们从零搭建了一个完整的客服Agent。现在回顾一下：我们做了哪些能力？它们如何组合在一起？一个生产级的客服Agent架构应该是什么样？

先回顾整个系列的能力地图：

基础能力：初始化、System Prompt、意图识别、知识库

感知能力：对话管理、情感分析

执行能力：订单查询、Function Calling、多Agent

智能能力：多轮进阶、向量检索、用户记忆

运营能力：效果评估、持续优化

图：客服Agent能力发展路线图

1

生产级客服Agent的完整架构

把所有能力整合起来，形成一个分层的架构：

图：生产级客服Agent完整架构

四层架构：

接入层

：Web/APP/小程序 → API网关，统一入口、负载均衡

Agent层

：路由调度 + 多Agent协作，每个Agent专注一个领域

能力层

：插件系统、Function Calling工具、记忆系统

数据层

：MySQL（业务数据）、向量库（知识库）、Redis（缓存/会话）

2

核心模块与技术栈

模块

技术选型

作用

对话引擎

大模型API（GPT-4o/Qwen等）

理解和生成

意图识别

关键词+LLM混合

识别用户意图

情感分析

关键词+LLM混合

识别用户情绪

知识库

Chroma/Milvus + Embedding

语义检索

工具调用

Function Calling

调用外部能力

数据库

MySQL/PostgreSQL

持久化存储

缓存

Redis

会话、热点数据

日志监控

ELK/Prometheus

可观测性

3

部署与运维

容器化部署：

yaml

```
# docker-compose.yml
```

```
version
```

```
:
```

```
"3.8"
```

```
services
```

```
:
```

```
agent
```

```
:
```

```
build
```

```
:
```

```
.
```

```
ports
```

```
:[
```

```
"8000:8000"
```

```
]
```

```
depends_on
```

```
:
```

```
mysql
```

redis

chroma

mysql

:

image

:

mysql:8.0

environment

:

MYSQL_ROOT_PASSWORD

:

xxx

volumes

:

"mysql_data:/var/lib/mysql"

]

redis

:

image

:

redis:7

volumes

:

"redis_data:/data"

]

chroma

:

image

:

chromadb/chroma:latest

volumes

: [

"chroma_data:/chroma"

]

关键运维要点：

健康检查：/health 接口监控核心依赖

日志采集：对话日志、错误日志、性能日志

告警：错误率、响应时长、转人工率异常时告警

自动扩缩容：根据QPS自动调整Agent实例数

灰度发布：新版本先灰度、再全量

4

安全与合规

数据加密

：传输用HTTPS，存储用AES加密

访问控制

：API Key、OAuth、RBAC

内容过滤

：输入输出都做敏感词过滤

越权防护

：用户只能访问自己的订单/数据

审计日志

：关键操作（删除、导出）记录审计

隐私合规

: GDPR/个保法，支持数据删除

5

一个完整的项目代码结构

bash

customer-service-agent/

|——

config/

配置文件

|

|——

base.yaml

|

|——

customer_service.yaml

|——

plugins/

插件

|

|——

intent_recognition/

|

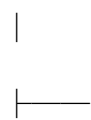
|——

sentiment_analysis/

|

|——

knowledge_base/



order_query/

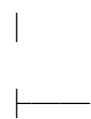


vector_knowledge/



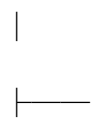
src/

核心代码



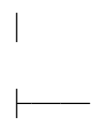
bot.py

主Bot



router.py

路由调度



dialog_manager.py

对话管理



memory.py

记忆系统

|

data/

数据

|

|

faq.json

|

|

intents.json

|

|

sentiments.json

|

tests/

测试

|

web/

Web界面

|

|

app.py

Flask后端

|

|

templates/

HTML模板

└──

docker-compose.yml

└──

requirements.txt

6

Web界面：让Agent真正可用

讲了这么多理论和代码，是时候让它"活"起来了。我们用 Flask 给客服Agent套上一个漂亮的Web外衣。

启动Web服务：

bash

cd

source-code

python3 web/app.py

浏览器访问 http://127.0.0.1:5000

Web界面实际运行效果：

图：客服Agent Web界面 – 多轮对话+情感识别+会话信息

界面核心功能：

实时对话

：用户输入消息，Bot即时回复

快捷按钮

：订单查询、退款咨询、订购花束、投诉建议

会话面板

：实时显示意图识别、对话轮数、响应时间、用户画像

对话重置

：一键清空历史，模拟新用户

API接口一览：

接口

方法

说明

/api/chat

POST

发送消息，返回Bot回复+会话信息

/api/status

GET

获取当前会话状态

/api/reset

POST

重置对话历史

/api/health

GET

健康检查

对话示例解读（看图说话）：

从截图可以看到几个典型场景：

多轮槽位填充

：用户说"订一束玫瑰花"→Bot问数量→用户说"99朵"→Bot确认预订

情感识别转人工

：用户说"我要投诉"→系统识别为愤怒情绪→立即转人工

FAQ知识检索

：用户问"如何退款"→系统返回完整的退款流程说明

右侧面板

：实时显示意图（order/complaint）、响应时间（3.7ms）、用户画像

7

项目源码与GitHub

本系列文章的完整代码已开源，欢迎 Star 关注！

GitHub: [fangxiao/customer-service-agent](https://github.com/fangxiao/customer-service-agent)

仓库内容包括：

完整的客服Agent代码实现（src/、plugins/）

14篇配套文章（公众号版HTML）

Web界面源码（Flask实现）

Mermaid图表生成脚本

配置文件、知识库、意图数据

快速开始：

bash

克隆仓库

```
git clone https://github.com/fangxiao/customer-service-agent.git
```

```
cd customer-service-agent/source-code
```

安装依赖

```
pip install -r requirements.txt
```

启动Web界面

```
python3 web/app.py
```

访问 <http://127.0.0.1:5000>

8

学习路线与进阶方向

恭喜！你已经具备了一个生产级客服Agent的完整知识体系。接下来可以进阶：

模型微调

：用业务数据微调，让模型更懂你的业务

Agent编排

：使用LangChain/LlamaIndex等框架

多模态

：支持语音、图片输入

主动对话

：Bot主动发起关怀、回访

智能体市场

：把你的Agent能力开放给其他系统

8

结语

客服Agent不是一个产品，而是一个

持续进化的生命体

。它的价值不在于"做了什么"，而在于"每天都在变好"。

回顾整个系列，你学到了：

从0到1搭建一个客服Agent的完整流程

插件化、模块化的设计思想

大模型时代的工程实践

数据驱动的迭代思维

希望这个系列能成为你在Agent开发路上的起点。未来，让我们一起见证更多AI Agent的精彩可能性。

—— 全系列完 ——

客服Agent实战系列

01 · 项目初始化

02 · Bot定义：System Prompt设计

03 · 意图识别插件

- 04 · 知识库插件
- 05 · 对话管理
- 06 · 情感分析插件
- 07 · 订单查询插件
- 08 · 多轮对话进阶
- 09 · 知识库增强与向量检索
- 10 · Function Calling
- 11 · 多Agent协作
- 12 · 用户记忆与个性化
- 13 · 效果评估与持续优化
- 14 · 生产级架构总结 ← 你在这里